NetBox ISC DHCP – manual

Plugin for NetBox 4.7+ that builds the ISC DHCP server configuration from NetBox data. Manual version: v0.10.0

1. What the plugin does

From NetBox objects (tenants, prefixes, DNS servers) the plugin builds ISC DHCP files: the main `dhcpd.conf`, class and static host tables for tenant groups, and subnet files per pool type. Files are stored ready-made (with a SHA256 hash) and recomputed when the data changes. The DHCP server downloads them with its own script.

2. Installation and update

```
sudo bash -c 'source /opt/netbox/venv/bin/activate && pip install --upgrade --force-reinstall --no-deps "git+https://github.com/ArK761/NetBox_ISC-DHCP.git@<COMMIT>" && cd /opt/netbox/netbox && python manage.py migrate netbox_isc_dhcp && systemctl restart netbox netbox-rq'
```

Enable the plugin in `configuration.py`: `PLUGINS = [..., "netbox_isc_dhcp"]`. Version and commit are shown top right on every plugin page.

3. Menu

Item	Content
Plugin CFG	Language (Slovak / English), path of `dhcpd.conf`, base path of files, subdirectories.
DHCP-CFG	Hub: Base dhcpd.conf, DNS servers, Subnets, Hosts and MAC addresses, Own variables.

Values are edited by clicking on them (a field and a Save button appear).

4. Plugin settings (Plugin CFG)

- **dhcpd.conf path** – directory on the DHCP server, e.g. `/etc/dhcp`.
- **Base path of files** – e.g. `/opt/dhcp`; subdirectories live below it.
- **Subdirectories** – e.g. `TABLE`, `SUBNET`; giving `/opt/dhcp/TABLE`.
- **Default subdirectory for tables** and **subdirectory for subnets**.

Everything is empty by default – the user sets it.

5. Base dhcpd.conf

- **Static part** – global DHCP server settings (authoritative, lease times, logging, primary subnet). **It must be written by the user**; the plugin never changes it.
- **Dynamic part** – the plugin writes **only INCLUDE lines** into it (paths of table and subnet files), never configuration.
- The **Preview** tab shows the whole resulting file.

6. DNS servers

Table Name + IP + **In use** (checkbox). At most 2 can be in use at once. If none is in use, the plugin shows a warning. DNS servers in use are available in pool code as `{{ dns }}`. All DNS servers (also unused) can be used in Own variables.

7. Subnets and pools

Subnet source

1. Choose the **object** (e.g. IPAM › Prefix) and the **subdirectory for subnets**.
2. **Add field** – any field of the object (standard or custom field).
3. Click a row – the list of all values opens with search, **Select all** / **Untick all** and the switch **only ticked** / **all except ticked**. Nothing ticked = *any (no filter)*.
4. On the field that defines the pool type (e.g. `POOL_TYPE`) click the **key** icon. A field with more than 100 values cannot be chosen.

Below the conditions you see the number of selected objects and the **gateway (GW)** for all prefixes: first or last usable IP.

Pools and their code

Every pool type has its own **code (template)**, a **Generate** switch and a **preview** on a real prefix. Variables in the code:

Variable	Meaning
`{{ network }}`, `{{ netmask }}`, `{{ prefix }}`	network address, mask, prefix (e.g. 10.0.0.0/24)
`{{ gateway }}`	gateway by the global setting
`{{ first_usable }}`, `{{ last_usable }}`	first / last usable address (never network or broadcast)
`{{ ip(n) }}`	n>0 from the network address, n<0 from the broadcast (−1 = last usable)
`{{ dns|join(",") }}`	DNS servers in use
`{{ cf.FIELD }}`, `{{ obj }}`	custom fields and the object itself

Example:

```
subnet {{ network }} netmask {{ netmask }} {
    USE LEASE_600
    option routers {{ gateway }};
    option subnet-mask {{ netmask }};
    pool {
        range {{ ip(2) }} {{ last_usable }};
    }
}
```

> If the code does not fit a prefix (e.g. ranges for /24 used on /28), the preview shows a red error and such a file is not used.

8. Own variables

Named pieces of ISC configuration. When adding one, pick an ISC option (lease, DNS, network, access, PXE, DDNS, own text), write a description and set the value right away:

Option	Value	Inserts
default + max lease	600	`default-lease-time 600;` `max-lease-time 780;` (factor 1.3 – configurable)
domain-name-servers	ticked DNS servers	`option domain-name-servers …;`
allow / deny duplicates	deny	`deny duplicates;`

In pool code type `..` – a list of variables appears at the cursor; pick with the arrows and Enter. A line `USE <name>` is inserted and replaced by the real lines in the result. A variable in use cannot be deleted. When renamed, the new name is written automatically into every code that uses it (USE lines).

9. Hosts and MAC addresses (tenants)

- Tenant group = DHCP class; tenant name = host-name.
- Every group has files (e.g. `Group_Table`, `Group_StaticTable`) – they can be renamed, added, included / left out of the config and assigned a subdirectory.
- For a file, tick the tenant custom fields holding MAC addresses (1–3) and the field with the static IP.
- A tenant without MAC is skipped; with a static IP it goes to the static table (with several MACs `NAME_1`, `NAME_2`…).
- Files are stored ready-made and recomputed when a tenant changes; the circular arrow icon recomputes them manually.

10. Troubleshooting

Symptom	Cause / fix
Red DNS warning	Switch on at least one DNS server.
Pools have number names	The key is on the wrong field – put it on the pool type field.
"Selected: 0 objects"	Check the conditions; nothing ticked means the field does not filter.
Error in pool preview	The code cannot be used for that prefix – adjust ranges (use `ip(n)`, `last_usable`).

11. Pool code check

On every save the plugin tries the code on **all prefixes of the pool**. If it finds an error, the code is **not saved** (the last working code stays), the errors are listed with the number of prefixes and an example, and your text stays in the editor. Checked:

- everything is inside one `subnet <network> netmask <mask> { }` block, network and mask match the prefix,
- `;` at the end of statements, closed quotes, parentheses and `{ }`,
- unknown or misspelled statements and options (standard ISC options + own options from the static part `option X code N`),
- the same option twice in one block,
- values: IP addresses (routers, DNS, NTP…), numbers (lease times), quoted text (domain-name, filename…),
- `option routers` on subnet level must be a usable address, `option subnet-mask` must match the mask, `default-lease-time` must not exceed `max-lease-time`,
- classes in `allow/deny members of "X"` must exist (tenant tables or static part),
- words in `log (...)` expressions – e.g. `foo` must be defined with `set foo = …` in the static part,
- `range` inside the network, not overlapping, gateway outside ranges, `ip(n)` within usable addresses, unknown `USE`.

> The check does not replace `dhcpd -t` on the DHCP server, but it catches the common errors that would stop the DHCP server from starting.

12. Pool sizes and incompatible subnets

- **Code is suitable for** – on save the plugin tries the code on sizes /16 to /30 and shows which work (e.g. /16 – /27).
- **Use for** – narrow where the code is used (e.g. only /24). It cannot exceed the suitable range; "auto" = the whole suitable range.
- A subnet whose size is outside the range (e.g. changed from /24 to /28 in NetBox) or that the code does not fit is **incompatible**: it is not exported and the DHCP server

keeps running without it.

- A red warning is then shown on every plugin page; the list with reasons is on the Subnets page and on the pool, and the pools table shows a red count.
- Incompatible subnets do not block saving the code – only subnets within the range are checked.